# OS AND REGEX

OS è una libreria standard di Python che serve per interagire con il sistema operative che permette di leggere file e cartelle, navigare tra directory, costruire percorsi di file e di gestire file automaticamente.

Nel progetto c'era bisogno di leggere dataset (file di testo), scrivere output (file con frasi segmentate), gestire più file.

Quindi os serve ad automatizzare la gestione dei file ed evitare di scrivere percorsi manualmente.

Una regex è un metodo per trovare o divider il testo attraverso l'uso di pattern. È uno strumento fondamentale in NLP perché lavora direttamente sulle stringhe.

Un' esempio estratto proprio dal progetto

re.split(r"[.!?]", text) il cui scopo è quello di dividere il testo quando trova "." , "!", "?" seguito da uno spazio

# FUNZIONI UTILIZZATE

**Import** serve a importare librerie e moduli (os e re).

L'istruzione if not os.path.exists(cartella_output): serve a controllare se la cartella di output esiste già. La funzione **os.path.exists()** restituisce un valore booleano: True se il percorso esiste, False se non esiste. L'uso di not inverte il risultato, quindi la condizione diventa vera se la cartella non è ancora presente. Questo controllo serve ad evitare errori nelle fasi di scrittura dei file.

os.makedirs(cartella_output) serve a creare la cartella di output. La funzione **makedirs** permette di costruire una directory e, se necessario, anche eventuali cartelle intermedie del percorso.

os.**listdir**(cartella_input) serve a ottenere la lista di tutti gli elementi contenuti nella cartella di input. Consente di scorrere tutto il contenuto della cartella invece di indicare i file uno a uno manualmente, ovvero rende il programma scalabile e applicabile a un intero insieme di documenti.

La struttura **for nome_file in lista_file**: è un ciclo for, cioè un'istruzione che permette di iterare su tutti gli elementi di una sequenza. Il ciclo prende, uno alla volta, i nomi contenuti nella lista dei file letti dalla cartella di input. A ogni iterazione la variabile nome_file assume il valore del file corrente, così il programma può controllarlo, aprirlo, elaborarlo e salvarne il risultato. Questo tipo di ciclo è essenziale quando si vuole applicare la stessa procedura a molti file in modo ripetuto e automatico.

La funzione nome_file.**endswith**(".sent_split") è un metodo delle stringhe che controlla se una stringa termina con una determinata sequenza di caratteri. Qui viene usata per verificare se il nome del file finisce con l'estensione .sent_split. Lo scopo è filtrare solo i file rilevanti per il task, ignorando eventuali altri file presenti nella cartella.

os.**path.join**(cartella_input, nome_file) e os.path.join(cartella_output, nome_file) servono a costruire i percorsi completi dei file di input e output. La funzione **join** unisce in modo corretto i vari componenti del percorso, adattandosi automaticamente al sistema operativo in uso. Questo è molto importante perché Windows usa il backslash \, mentre Linux e macOS usano lo slash /.

**open**(percorso_input, "r", encoding="utf-8") serve ad aprire il file di input in modalità lettura. La modalità "r" indica che il file deve essere letto e non modificato, mentre encoding="utf-8" specifica la codifica dei caratteri, permettendo di gestire correttamente lettere accentate, simboli e testi multilingui. La funzione open restituisce un oggetto file.

open(percorso_output, "w", encoding="utf-8") apre il file di output in modalità scrittura. La modalità "w" significa che il file verrà creato se non esiste, oppure sovrascritto se esiste già. Anche qui l'encoding UTF-8 è usato per gestire correttamente i caratteri testuali. Questa chiamata è il punto in cui il programma prepara il salvataggio del risultato: dopo aver letto, segmentato e pulito il testo, crea il file destinato a contenere le frasi finali.

file_input.**read**() è il metodo che legge tutto il contenuto del file in una sola volta e lo restituisce come stringa. Questa scelta è adatta quando i file non sono troppo grandi e quando si vuole lavorare sull'intero contenuto testuale con operazioni globali, come sostituzioni o segmentazioni. In questo script ad esempio tutto il processo viene effettuato sulla stringa completa.

file_input.**close**() serve a chiudere il file dopo la lettura. Quando un file viene aperto, il sistema operativo alloca risorse per mantenerlo accessibile; con close() quelle risorse vengono liberate.

testo.**replace**("<EOS>", " ") è un metodo delle stringhe che sostituisce tutte le occorrenze di una sottostringa con un'altra. In questo caso ogni token speciale <EOS>, che rappresenta la fine di una frase, viene rimpiazzato con uno spazio. Serve a ripulire il testo da questo marcatore prima di applicare la segmentazione basata sulla punteggiatura.

re.**split**(r'(?<=[.!?])\s+', testo) è una delle istruzioni più importanti del programma, perché realizza la vera segmentazione in frasi. Split divide una stringa in base a un pattern. La parte (?<=[.!?]) significa che la divisione deve avvenire solo se prima c'è un punto, un punto esclamativo o un punto interrogativo. La parte \s+ indica invece uno o più spazi bianchi. In pratica il testo viene spezzato nei punti in cui, dopo un segno di punteggiatura terminale, compare uno spazio.

frasi_finali = [] inizializza una lista vuota che serve come contenitore delle frasi finali. Dopo essere state ottenute con la regex, verranno pulite e validate, verranno così inserita in questa lista inizialmente vuota.

Il secondo ciclo **for pezzo in pezzi**: serve a scorrere tutti i segmenti prodotti dalla re.split. Ogni pezzo rappresenta una possibile frase, ma potrebbe contenere spazi inutili o addirittura essere vuoto. Il ciclo permette quindi di applicare a ciascun elemento una fase di pulizia.

pezzo.**strip**() agisce sulle stringhe rimuovendo gli spazi bianchi all'inizio e alla fine della stringa, compresi caratteri come spazi, tabulazioni e newline. Questo significa che una sequenza come " Ciao! \n" diventa semplicemente "Ciao!".

La condizione **if pezzo_pulito != "":** controlla che la stringa pulita non sia vuota e si eliminano casi indesiderati.

frasi_finali.**append**(pezzo_pulito) aggiunge un elemento in fondo ad una lista.

file_output.**write**(frase + "\n") è il metodo che scrive il contenuto nel file. Per ogni frase contenuta in frasi_finali, il programma scrive la stringa della frase e poi aggiunge il carattere \n, che rappresenta l'andata a capo, ciò permette di avere una frase per riga nel file finale.

file_output.**close**() chiude il file di output dopo che tutte le frasi sono state scritte.

**print**("Creato file:", nome_file) usa la funzione print() per mostrare un messaggio a schermo.

La funzione file**.readlines**() è un metodo che permette di leggere il contenuto di un file suddividendolo automaticamente in una lista di righe. Quando viene chiamata restituisce una lista in cui ogni elemento corrisponde a una riga del file comprensiva del carattere di newline \n alla fine. A differenza di read(), che restituisce una singola stringa contenente tutto il testo, readlines() mantiene già una struttura a lista che può essere iterata facilmente con un ciclo for.

La funzione **set**() crea un insieme a partire da una sequenza, come una lista. Un insieme è una struttura dati che contiene elementi unici, cioè elimina automaticamente i duplicati, e non mantiene un ordine specifico degli elementi. Inoltre, i set permettono operazioni matematiche efficienti come l'intersezione, che sono fondamentali per calcolare le metriche di valutazione.

Il metodo set.**intersection**() è un'operazione definita sugli insiemi che restituisce un nuovo insieme contenente solo gli elementi comuni tra due set. set_gold.intersection(set_pred), restituisce tutte le frasi che compaiono sia nel gold sia nella predizione.

La funzione **len**() restituisce il numero di elementi contenuti in una struttura dati, come una lista, un set o una stringa. Utilizzata per trasformare strutture complesse in valori numerici su cui poter fare calcoli. Ad esempio, len(set_gold.intersection(set_pred)) restituisce il numero di frasi in comune (veri positivi), mentre len(set_pred) e len(set_gold) restituiscono rispettivamente il numero totale di frasi predette e gold.

La funzione **round**() serve ad arrotondare un numero a un certo numero di cifre decimali. La sua sintassi è round(numero, n), dove n indica quante cifre mantenere dopo la virgola.

La funzione os.**path.exists**()verifica se un determinato percorso (file o cartella) esiste nel filesystem. Restituisce un valore booleano: True O False; fondamentale per evitare errori di esecuzione, perché aprire un file inesistente causerebbe problemi.

L'accesso ai dizionari tramite la **sintassi r["precision"]** non è una funzione ma un'operazione fondamentale di Python per lavorare con i dizionari. Un dizionario è una struttura dati che associa chiavi a valori, e questa sintassi permette di recuperare il valore corrispondente a una determinata chiave.

La funzione **str**() serve a convertire un valore in stringa. Rende stampabile e combinabile con altre stringhe un valore numerico. Usato per numerare ordinatamente le frasi non matchate, producendo output come 1. frase, 2. frase, 3. frase. Quindi il suo ruolo non è matematico ma di formattazione del testo, ed è molto utile quando vuoi costruire messaggi leggibili e ben presentati.

L'assegnazione **contatore = contatore + 1** serve ad aumentare di uno il valore della variabile contatore a ogni iterazione. Anche questa non è una funzione, ma è una operazione fondamentale perché permette di numerare progressivamente le frasi stampate. All'inizio contatore vale 1, poi dopo aver stampato la prima frase diventa 2, poi 3, e così via. È un meccanismo di incremento, per aggiornare una variabile in base al suo valore precedente.

# SPIEGAZIONE SEGUENDO LO SCRIPT

## Sentence_split_versione1(punteggiatura).py

```
import os
import re
# OS è una libreria standard di Python che serve per interagire con il sistema operative che
permette di leggere file e cartelle, navigare tra directory, costruire percorsi di file e di
gestire file automaticamente.
Nel progetto c'era bisogno di leggere dataset (file di testo), scrivere output (file con frasi
segmentate), gestire più file.
Quindi os serve ad automatizzare la gestione dei file ed evitare di scrivere percorsi
manualmente.
Una regex è un metodo per trovare o divider il testo attraverso l'uso di pattern. È uno
strumento fondamentale in NLP perché lavora direttamente sulle stringhe.
Un' esempio estratto proprio dal progetto
re.split(r"[.!?]", text)  il cui scopo è quello di dividere il testo quando trova  "." , "!", "?"
seguito da uno spazio


cartella_input = "data/raw"
cartella_output = "output/sentence_split_versione1"
#Definisco le cartelle di input ed output attraverso variabili di modo che se devo
modificare il percorso modifico solo qua.

if not os.path.exists(cartella_output):
   os.makedirs(cartella_output)
#Se la cartella output non esiste allora con la funzione interna di os makedirs la creo.

lista_file = os.listdir(cartella_input)
#Definisco la variabile per memorizzare in una lista (con la libreria os e la funzione listdir)
tutti i file della cartella di input.

for nome_file in lista_file:
   if nome_file.endswith(".sent_split"):
#ciclo for esterno in cui cerco solo i file che finiscono con ".sent.split"

      percorso_input = os.path.join(cartella_input, nome_file)
      percorso_output = os.path.join(cartella_output, nome_file)
#definisco e costruisco il percorso completo dei file di input ed output (attraverso due
variabili), quindi dove si trovano i file gold e i file predittivi di output

      file_input = open(percorso_input, "r", encoding="utf-8")
      testo = file_input.read()
      file_input.close()
#open è una funzione con 3 parametri che apre file ed i 3 parametri sono :  percorso,
modalità di apertura (qui è read) e tipologia di codifica del testo. E restituisce un oggetto
file (file_input)
Dopodiché definisco la variabile testo come stringa.
Infine chiudo il file (potevo usare anche "with open(percorso_input, "r", encoding="utf-8")
as file_input:" che  avrebbe chiuso automaticamente il testo e non serviva la riga finale.

      testo = testo.replace("<EOS>", " ")
```

```
#Modifico la versione testo con la funzione replace che cerca tutte le occorrenze di <EOS> e le sostituisce con uno spazio vuoto “ “.

    pezzi = re.split(r'(?<=[.!?])\s+', testo)
#Qui utilizzo la funzione split di regex er spezzare il testo in una lista din frasi attraverso la regola “(?<=[.!?])\s” ovvero è come dire:  “ se trovi un punto o un punto esclamativo o interrogativo seguiti da almento uno spazio (\s+) allora dividi qua”.  “ r’ ” all’inizio serve a classificare \s non come parte del testo da cercare ma come spazio vuoto.

    frasi_finali = [ ]
#inizializzo una lista vuota  che utilizzerò nel ciclo for

    for pezzo in pezzi:
      pezzo_pulito = pezzo.strip()
#itero per ogni elemento di pezzi la funzione .strip che elimina spazi iniziale e finali e newline \n
      if pezzo_pulito != "":
        frasi_finali.append(pezzo_pulito)
#controllo che la stringa non sia vuota senoo la scarto, potrei avere falsi errori quando faccio la valutazione. Passato questo controllo con .append la aggiungo alla lista.

    file_output = open(percorso_output, "w", encoding="utf-8")
# prearo il file di output con la funzione open che questa volta è in scrittura (“w”).

    for frase in frasi_finali:
      file_output.write(frase + "\n")
#ciclo for su tutte le frasi finali e le scriviamo nel file_output aggiungendo una newline (\n)

    file_output.close()
# chiudo il file dopo la scrittura

    print("Creato file:", nome_file)
# stampo “creato file: e nome file”
Qua si chiude sia l’ultimo cicllo for interno che quello esterno iniziale.

print("Ho finito.")
# stampo la fine del programma.
```

## Valutazione_versione1.py

```
import os
#importo modulo os per interagire con i file, cartelle e percorsi
cartella_gold = "data/processed"
cartella_pred = "output/sentence_split_versione1"
#definisco variabili per confrontare file processati predittivi dallo script con i file gold.
def leggi_frasi(percorso):
  file = open(percorso, "r", encoding="utf-8")
  righe = file.readlines()
  file.close()
```

# definisco una funzione con parametro la variabile percorso del file da aprire. Questa apre il file in lettura e legge tutto il contenuto restituendo una lista di righe separate da newline. #Quindi viene salvato nella variabile righe che sarà poi percorsa per ripulire tutte le righe mantenendo solo righe utili.

```
  frasi = []
   for riga in righe:
      riga = riga.strip()
      if riga != "":
         frasi.append(riga)
```

#creo una lista vuota che verrà riempita dalle frasi valide.

#Il ciclo for serve a scorrere ogni riga della lista righe per poi pulire da spazi vuoti e newline tabulazioni etc. Viene quindi normalizzata la lista. Aggiungendo if riga !="" posso controllare che la riga non sia vuota per poi aggiungerla all'elenco frasi con la funzione append.

```
return frasi
```

#Concludo la funzione  leggi_frasi con Return, di modo che il tutto sia un valore assegnabile o scrivibile a variabili esterne alla funzione leggi_frasi.

```
def divisione_sicura(a, b):
   if b == 0:
      return 0.0
   return a / b
```

# serve ad effettuare divisioni evitando lo zero al denominatore visto che dopo utilizzeremo rapporti per effettuare la valutazione. La funzione riceve due parametri a e b, se b coincide con 0 allora dice di riportare come valore 0 altrimenti la frazione a/b. Da qui possiamo fare conti con più tranquillità.

```
lista_file = os.listdir(cartella_gold)
```

#restituisce la lista di tutti gli elementi della cartella gold e li salva in lista_file. Trasforma un insieme di file in una lista di nomi lavorabili in sequenza.

```
tutti_risultati = []
```

#creo lista vuota per poter poi inserire i risultati

```
for nome_file in lista_file:
   if nome_file.endswith(".sent_split"):
```

#per ogni elemento della lista_file da il via alle successive funzioni solo se termina in ".sent_split"

```
      percorso_gold = os.path.join(cartella_gold, nome_file)
```

```
    percorso_pred = os.path.join(cartella_pred, nome_file)
#definisco variabili per i percorsi specifici interi delle cartelle gold e predittivi

    if not os.path.exists(percorso_pred):
        print("Manca file predetto:", nome_file)
        continue
#verifica l'esistenza del file predittivo corrispondente, se non è cosi (quindi TRUE) allora stampa la mancanza e poi va avanti

    frasi_gold = leggi_frasi(percorso_gold)
    frasi_pred = leggi_frasi(percorso_pred)
#applico la funzione personalizzata leggi_frasi sia al percorso gold che pred, che serve a leggere e pulire i file e poi li assegna alle due variabili corrispettive

    set_gold = set(frasi_gold)
    set_pred = set(frasi_pred)
#La funzione set trasforma le due liste in un insieme. Gli insiemi servono per avere elementi unici senza duplicati e supportano operazioni matematiche. Bisogna però tenere conto che se una frase esiste due volte viene contata una sola. E non conta cosi la posizione della frase.

    veri_positivi = len(set_gold.intersection(set_pred))
#la funzione len conta i numero di elementi di una struttura dati. Intersection calcola l'intersezione tra due insiemi, in questo caso set gold e pred. Serve ad intercettare i match esatti che sono usciti dallo script ovvero i veri positivi.

    tot_pred = len(set_pred)
    tot_gold = len(set_gold)
#calcolo il numero di frasi predette ed il numero di frasi gold

    precision = divisione_sicura(veri_positivi, tot_pred)
    recall = divisione_sicura(veri_positivi, tot_gold)
#per precision e recall utilizzo la funzione divisione_sicura come rapporto tra veri positivi e tot predetto e veri positivi per la precision e tot gold per la recall.
    if precision + recall == 0:
        f1 = 0.0
    else:
        f1 = 2 * precision * recall / (precision + recall)
#questa righe servono a gestire nel calcolo della f1 divisioni per zero, essendo f1 una divisione per precision + recall, se sono entrambe 0 darà 0 altrimenti darà f1.
```

```
    risultato = {
        "precision": precision,
        "recall": recall,
        "f1": f1
    }
```

#risultato diventa un dizionario che associa chiavi e valori, le chiavi sono le stringhe precision, recall e f1 mentre i valori sono quelli appena calcolati.

```
    tutti_risultati.append(risultato)
```

#con la funzione append costruisco una collezione di metriche. Mi servira per una visione globale.

```
    print("\nFile:", nome_file)
    print("Frasi gold     :", tot_gold)
    print("Frasi predette :", tot_pred)
    print("Match esatti   :", veri_positivi)
    print("Precision      :", round(precision, 4))
    print("Recall         :", round(recall, 4))
    print("F1             :", round(f1, 4))
```

#stampo tutti i risultati vari. All'interno trovo round che arrotonda i numeri a 4 cifre decimali. La utilizzo in stampa non in calcolo.

```
if len(tutti_risultati) > 0:
  somma_precision = 0
  somma_recall = 0
  somma_f1 = 0
```

# controlla se almeno un elemento è stato calcolato attraaverso la funzione len. Ed inizializzo le somme uguali a zero.

```
  for r in tutti_risultati:
    somma_precision += r["precision"]
    somma_recall += r["recall"]
    somma_f1 += r["f1"]
```

# per ogni somma utillizzo r ["…"] che serve a recuperare dal dizionario r i valori associati alle rispettive chiavi . "+=" che somma al valore di sinistra iniziale il valore di destra (quello preso dal dizionario).

```
 media_precision = somma_precision / len(tutti_risultati)
  media_recall = somma_recall / len(tutti_risultati)
  media_f1 = somma_f1 / len(tutti_risultati)
```

#calcolo le medie dividendo le varie somme rispetto il totale di tutti i file valutati.

```
    print("\n===== MEDIA COMPLESSIVA =====")
    print("Precision media:", round(media_precision, 4))
    print("Recall media   :", round(media_recall, 4))
    print("F1 media       :", round(media_f1, 4))
else:
    print("Nessun file valutato.")
```

## Debug_versione1.py

```
import os

cartella_gold = "data/processed"
cartella_pred = "output/sentence_split_versione1"

def leggi_frasi(percorso):
    file = open(percorso, "r", encoding="utf-8")
    righe = file.readlines()
    file.close()
#definisco funzione leggi_frasi il cui parametro è il percorso del file da leggere che aprirà
un file del percorso in modalità lettura ed assegno a righe una lista di stringhe estratte dal
file e poi lo chiude.

    frasi = []
# creo una lista vuota in cui poi inseriremo le frasi pulite senza spazi e righe vuote.

    for riga in righe:
        riga = riga.strip()
        if riga != "":
            frasi.append(riga)
# per ogni elemento del file applico la funzione strip che toglie spazi iniziali e finali e
newline. Se la riga non è vuota allora con append la aggiunge all'elenco frasi.

    return frasi
#restituisce la lista frasi in modo da poterla richiamare in futuro.

lista_file = os.listdir(cartella_gold)
#legge i file gold nella cartella gold

for nome_file in lista_file:
    if nome_file.endswith(".sent_split"):
#per ogni file trovato controlla se termina nel modo giusto e prosegue

        percorso_gold = os.path.join(cartella_gold, nome_file)
        percorso_pred = os.path.join(cartella_pred, nome_file)
#definisco il percorso completo dei file gold e predittivi

        if not os.path.exists(percorso_pred):
            print("Manca file predetto:", nome_file)
            continue
```

```
#controllo se il file predetto corrispondente esiste se non esiste continua comunque
andare in errore), ma stampa la mancanza

    frasi_gold = leggi_frasi(percorso_gold)
    frasi_pred = leggi_frasi(percorso_pred)

    set_gold = set(frasi_gold)
    set_pred = set(frasi_pred)
#con la funzione set trasformo la lista in un insieme senza duplicati e sul quale si possano
fare operazioni sugli isnsiemi

    frasi_mancate = set_gold - set_pred
#prende tutte le frasi mancate sottraendo i due insiemi

    print("\n==============================")
    print("FILE:", nome_file)
    print("==============================")

    print("\nFrasi NON trovate dalla Versione 1:")
# stampo tutti i risultati (sono ancora dentro il ciclo forn nome_file in lista_file)

    if len(frasi_mancate) == 0:
      print("Nessuna")
    else:
      contatore = 1
# se il numero di frasi mancate è zero stamapa nessuna, attraverso len conto gli elementi di
frasi mancate. Imposto il contatore a 1

      for frase in frasi_mancate:
        print(str(contatore) + ".", frase)
        contatore = contatore + 1
# per ogni frase in frasi mancate stampo il numero e con str lo trasfformo in stringa e cosi
aggiungo 1 al contatore e da qui ricomincio il ciclo.
```

## Confronto_errori_versione1_versione2.py

```
import os
# modulo os, che serve per lavorare con file, cartelle e percorsi del sistema operativo.

cartella_gold = "data/processed"
cartella_v1 = "output/sentence_split_versione1"
cartella_v2 = "output/sentence_split_versione2"
# definisco tre variabili stringa che contengono i percorsi delle cartelle da usare nel
confronto. cartella_gold contiene i file corretti di riferimento, mentre cartella_v1 e
cartella_v2 contengono gli output prodotti rispettivamente dalla versione 1 e dalla versione
2.

def leggi_frasi(percorso):
  file = open(percorso, "r", encoding="utf-8")
  righe = file.readlines()
  file.close()
# definisco una funzione chiamata leggi_frasi il cui argomento è un percorso di file. Lo
scopo della funzione è leggere quel file e restituire una lista di frasi pulite. In questo modo
puoi riutilizzare la stessa procedura per leggere gold, versione 1 e versione 2 senza
```

riscrivere sempre lo stesso codice. Con Open apro I file in modalità lettura, e per ogni file leggo riga per riga il cui risultato è una lista di stringhe dove ogni element è una riga del file. Dopodiche chiudo il file

```
    frasi = []
#inizializzo una nuova lista vuota per raccogliere le varie frasi

    for riga in righe:
        riga = riga.strip()
        if riga != "":
            frasi.append(riga)
# inizio un ciclo for che scorre tutte le righe e con la funzione strip pulisco da spazi iniziali, finali e newline.
#se la riga è non vuota allora applico append che aggiunge la riga alla lista frasi.

    return frasi
# restituisce all'esterno del ciclo la lista finale delle sole frasi buone.

lista_file = os.listdir(cartella_gold)
#restituisce la lista di file della cartella gold. Utilizzata per confrontare poi i file

for nome_file in lista_file:
    if nome_file.endswith(".sent_split"):
# ciclo for che scorre i file presenti in lista_file ed ad ogni iterazione contiene il nome del file corrente. Va avanti nel ciclo solo se il file termmina con .sent_split

        percorso_gold = os.path.join(cartella_gold, nome_file)
        percorso_v1 = os.path.join(cartella_v1, nome_file)
        percorso_v2 = os.path.join(cartella_v2, nome_file)
# Costruisco I percorsi completi dei tre file da confrontare: il gold, l'output della versione 1 e l'output della versione 2. os.path.join per unire correttamente cartella e nome file.
# path.join serve a costruire I percorsi in modo giusto su qualsiasi piattaforma.

        if not os.path.exists(percorso_v1):
            print("Manca file versione1:", nome_file)
            continue
        if not os.path.exists(percorso_v2):
            print("Manca file versione2:", nome_file)
            continue
#se il file della versione 1 non corrisponde alla versione gold stampa su schermo e va avanti. Stessa cosa per la versione 2.

        frasi_gold = leggi_frasi(percorso_gold)
        frasi_v1 = leggi_frasi(percorso_v1)
        frasi_v2 = leggi_frasi(percorso_v2)
#Richiamo la funzione leggi_frasi per leggere e pulire le frasi del gold, della versione 1 e della versione 2. Ottenendo quindi tre liste di frasi pronte per il confront

        set_gold = set(frasi_gold)
        set_v1 = set(frasi_v1)
        set_v2 = set(frasi_v2)
#Qui trasformi le tre liste in insiemi (set). Questo elimina eventuali duplicati e permette di usare operazioni insiemistiche come la differenza (applichiamo subito dopo).

        mancate_v1 = set_gold - set_v1
```

```
    mancate_v2 = set_gold - set_v2
#Calcolo le frasi mancanti per ciascuna versione. Il risultato sono gli errori di recall, cioè le frasi corrette che il sistema non ha recuperato.

    print("\n==============================")
    print("FILE:", nome_file)
    print("==============================")
#stampo un intestazione per separare i risultati dei diversi file. \n aggiunge una riga vuota prima della linea di separazione. Questo rende l'output più leggibile.

    print("\nFrasi NON matchate da VERSIONE 1:")
    if len(mancate_v1) == 0:
      print("Nessuna")
    else:
      contatore = 1
      for frase in mancate_v1:
        print(str(contatore) + ".", frase)
        contatore = contatore + 1
#Controllo se il set delle frasi mancanti della versione 1 è vuoto. Se ha lunghezza zero, significa che la versione 1 non ha perso nessuna frase gold. In caso positivo stampo "Nessuna" altrimenti avvio un contatore a partire da 1. Inizio un nuovo ciclo per tutte le frasi mancate (con str trasformo il numero del contatore in stringa) ed aggiorno il contatore aggiungendo 1.

    print("\nFrasi NON matchate da VERSIONE 2:")
    if len(mancate_v2) == 0:
      print("Nessuna")
    else:
      contatore = 1
      for frase in mancate_v2:
        print(str(contatore) + ".", frase)
        contatore = contatore + 1
#faccio la stessa cosa per il v2
```

## Clean_dataset.py

```
import os
# Importo il modulo os, necessario per lavorare con cartelle e file.

cartella_input = "data/raw"
cartella_output = "data/processed"
# Definisco due variabili che indicano la cartella di input e la cartella di output. La prima contiene i file raw, cioè i dati grezzi; la seconda conterrà i file puliti, cioè il dataset processato.

if not os.path.exists(cartella_output):
  os.makedirs(cartella_output)

lista_file = os.listdir(cartella_input)
# Controllo se la cartella di output esiste già. Se non esiste, passo al blocco successivo per crearla, con listdir creo una lista di nomi dei file da lavorare.


for nome_file in lista_file:
```

```
  if nome_file.endswith(".sent_split"):
#per ogni file nella lista file lo filtro in base a come finisce

    percorso_input = os.path.join(cartella_input, nome_file)
    percorso_output = os.path.join(cartella_output, nome_file)
#do le diettive per i percorsi completi che con la funzione join viene creata bene per ogni
tipo di piattaforma.

    file_input = open(percorso_input, "r", encoding="utf-8")
    testo = file_input.read()
    file_input.close()
#apro i file in lettura e leggo tutto il testo come un'unica stringa e chiudo il file

    testo = testo.replace("<EOS>", "\n")
#sostituisco tutti i token <EOS> con una newline

    righe = testo.split("\n")
    frasi_pulite = []
#divido il testo in corrispondenza delle newline

    for riga in righe:
      riga = riga.strip()
      if riga != "":
        frasi_pulite.append(riga)
#per ogni elemento riga nella lista righe dopo lo split cerco le linee vuote e se non lo sono
le aggiungo alla lista frasi pulite.

    file_output = open(percorso_output, "w", encoding="utf-8")
#apro file di output in modalità scrittura

    for frase in frasi_pulite:
      file_output.write(frase + "\n")

    file_output.close()
#per ogni frase di frasi pulite itero scrivendola nel file di output e andando a capo passo
all'elemento successivo del ciclo for chiudendo cosi il file output.

    print("Creato file gold:", nome_file)

print("Pulizia dataset completata.")
```